NETWORK TRAFFIC FORENSICS

Investigation of Suspected STRRAT Malware Communication

PCAP Analysis | *Wireshark + Python (Scapy)*

Prepared by: Ankit Gawade

Source: malware-traffic-analysis.net | 30 July 2024 exercise

1. Executive Summary

A packet capture from a public malware-analysis exercise was examined using Wireshark and Python (Scapy). The script processed 11,562 packets and summarized IP activity, network conversations, destination ports, and DNS queries. Manual TCP-stream inspection revealed repeated structured messages containing the STRRAT identifier and host information, including the hostname, username, operating system, and installed security software.

Key Finding: Traffic between internal host 172.16.1.66 and external address 141.98.10.79 over TCP port 12132 contained repeated STRRAT-related messages. Timestamp analysis of 102 matching packets measured a median interval of 5.015 seconds (mean 5.026 seconds), providing quantitative evidence of regular automated check-ins consistent with suspected STRRAT command-and-control (C2) behavior. Further payload inspection revealed Base64-encoded application and window titles, including Documents, Pictures, and Program Manager, suggesting that user-activity information was also being transmitted to the external server.

Assessment: Suspicious / high-priority for further investigation. The findings support suspected STRRAT C2 communication involving host-information disclosure and possible user-activity monitoring. However, the available network evidence does not establish remote command execution, credential theft, file exfiltration, or the full compromise timeline.

2. Scope, Source and Limitations

Item	Details
Evidence source	Public PCAP: https://www.malware-traffic-analysis.net/2024/07/30/index.html
Analysis environment	Wireshark; Python with Scapy; Kali Linux
Packets reported	11,562
Investigated flow	172.16.1.66 ↔ 141.98.10.79, TCP 12132
Scope	Offline PCAP investigation including traffic triage, payload inspection and packet-level timing analysis; no live endpoint examined

The screenshots and packet totals originate from the investigator’s laboratory analysis of the public PCAP. The updated analysis also includes the investigator’s Scapy timestamp output and Wireshark filtered-packet screenshot. No endpoint artifacts, malware binary, file-exfiltration proof or command-execution evidence were examined in this report.

3. Investigation Methodology

- Triage: Review Wireshark IPv4 endpoints and Conversations statistics to understand communicating hosts and traffic volume.
- Automated summary: Use Scapy to count source/destination IPs, connection pairs, destination ports and DNS queries.
- Focused inspection: Examine the identified host pair with a Wireshark display filter and review associated TCP traffic.
- Payload validation: Use Follow → TCP Stream to inspect application data for characteristic strings and recurring messages.
- Beaconing measurement: Filter outbound packets containing ping|STRRAT; compare timestamps of consecutive matching packets and summarize the intervals.

· Documentation: Record observable indicators, distinguish evidence from interpretation and propose follow-up validation.

4. Initial Traffic Triage

Wireshark statistics provided a broad view of the capture before individual sessions were inspected. The images below reproduce the original IPv4 endpoint and conversation analysis.

Topic / Item	Cou	Average	Min Val	Max Val	Rate (ms)	Percent	Burst Rate	Burst Start
IPv4 Statistics/All Addresses	11531				0.0197	100%	13.6000	68.485
172.16.1.66	11531				0.0197	100.00%	13.6000	68.485
199.232.196.209	6540				0.0112	56.72%	13.6000	68.485
172.16.1.4	1713				0.0029	14.86%	1.6700	0.569
185.199.110.133	653				0.0011	5.66%	5.1100	65.341
141.98.10.79	411				0.0007	3.56%	0.0400	78.066
23.52.9.222	232				0.0004	2.01%	0.1700	9.426
204.79.197.203	194				0.0003	1.68%	0.2500	328.544
23.198.7.175	189				0.0003	1.64%	0.3800	28.875
23.198.7.177	173				0.0003	1.50%	0.7000	83.546
23.48.203.208	165				0.0003	1.43%	0.2700	36.478
20.189.173.26	121				0.0002	1.05%	0.7800	227.297
23.53.11.166	85				0.0001	0.74%	0.2300	329.396
52.109.20.47	81				0.0001	0.70%	0.4900	203.780
23.198.7.168	65				0.0001	0.56%	0.4000	329.085
52.113.194.132	63				0.0001	0.55%	0.4500	204.080
52.109.0.142	61				0.0001	0.53%	0.1300	29.820
20.189.173.10	59				0.0001	0.51%	0.4700	210.684
23.194.164.136	55				0.0001	0.48%	0.3400	340.876
40.126.29.14	48				0.0001	0.42%	0.1400	110.205
52.109.0.91	46				0.0001	0.40%	0.2800	203.531
23.46.192.165	44				0.0001	0.38%	0.1800	329.699
20.189.173.16	41				0.0001	0.36%	0.2400	0.693
52.109.6.53	37				0.0001	0.32%	0.0800	28.921
20.7.2.167	36				0.0001	0.31%	0.0700	35.313
172.16.1.255	34				0.0001	0.29%	0.0600	0.000

Display filter: Enter a display filter … Apply

Figure 1. Wireshark IPv4 statistics.

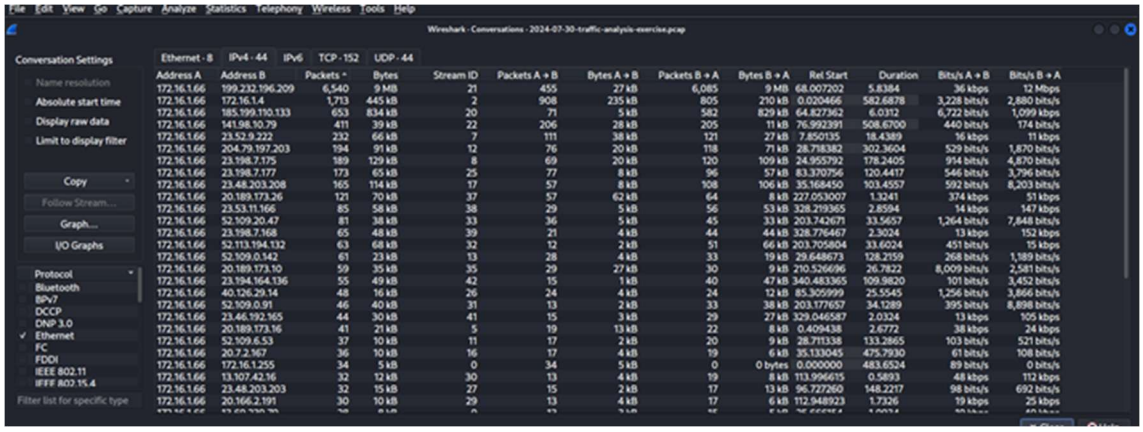


Figure 2. Wireshark conversation summary .

5. Python-Assisted PCAP Summary

The original Scapy script iterated through packets and collected IPv4 addresses, IP pairs, TCP/UDP destination ports and DNS question names, using collections.Counter to rank occurrences. This is a traffic inventory and prioritization aid, not a malware detector on its own.

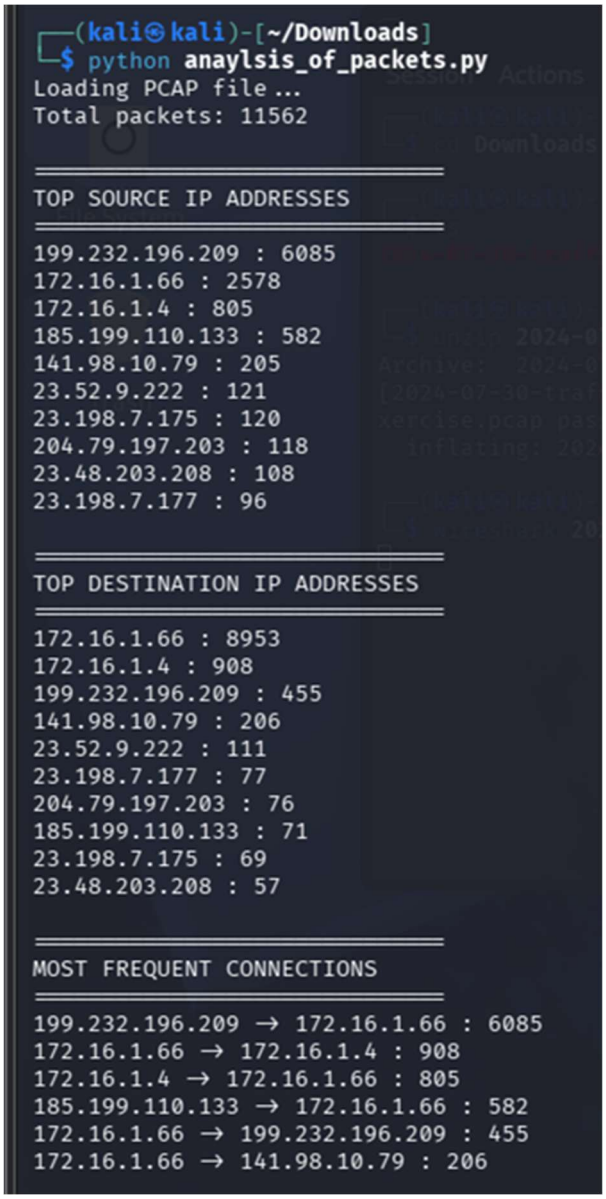


Figure 3. Scapy output: 11,562 packets and highest-frequency source/destination IPs.



Figure 4. Scapy output: conversations, destination ports and sample DNS queries.

The output includes 172.16.1.66 → 141.98.10.79 (206 packets in the script's directional IP-pair count) and destination port 12132 (206 occurrences). These counts describe packets matching the script's aggregation logic; they should not be presented as 206 separate sessions.

Triage rationale: The internal host 172.16.1.66 appeared in 2,578 source packets and 8,953 destination packets, making it a useful focus for traffic review. Its conversation with external host 141.98.10.79 accounted for 206 outbound and 205 inbound packets in the directional IP-pair counts. The script also counted 206 packets with destination port 12132. These observations made the host pair worth inspecting, but packet volume and an uncommon port alone do not establish maliciousness. Ports 49750–49754 also appear frequently in the output; because they lie in the dynamic/private port range, some may be ephemeral client ports in reply traffic (their precise roles would need verification from TCP endpoints). The decisive evidence for prioritizing 141.98.10.79 was the subsequently observed ping|STRRAT payload and its repeated timing pattern. These statistics provide a defensible triage rationale, not a claim about the undocumented exact order in which all candidate IPs were reviewed.

6. Focused Examination of Suspicious Communication

The original Wireshark investigation isolated traffic between internal address 172.16.1.66 and external address 141.98.10.79 with this display filter:

ip.addr == 172.16.1.66 && ip.addr == 141.98.10.79

The investigated traffic included outbound TCP destination port 12132. In contrast, destination ports 49750–49754 fall in the dynamic/private range commonly used for temporary client endpoints; their high counts do not by themselves indicate suspicious services. Port 12132 is not proof of malware either. The recurring structured STRRAT payload and measured timing provide the stronger basis for the finding.

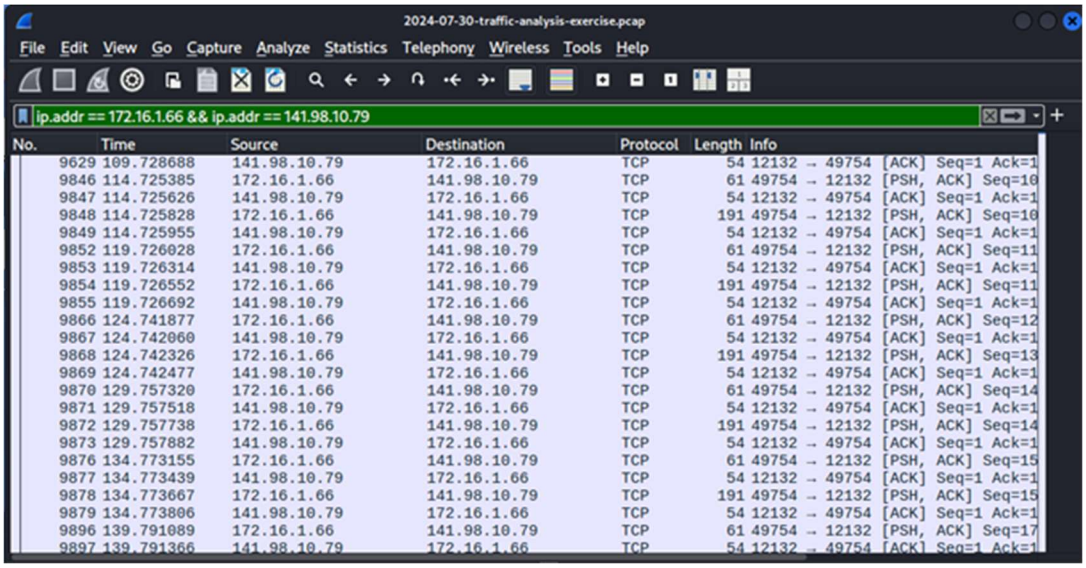


Figure 5. Wireshark display filter for the host pair and resulting TCP traffic.

The original analyst then used Follow → TCP Stream to inspect the connection's application data.

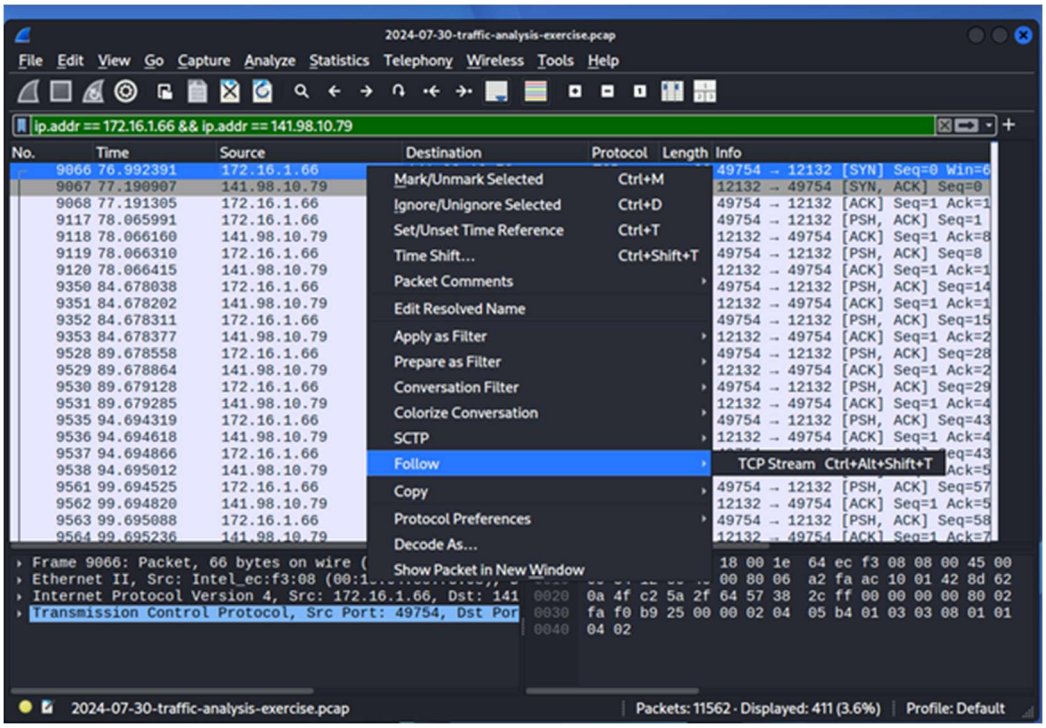


Figure 6. Selecting Follow → TCP Stream in Wireshark.

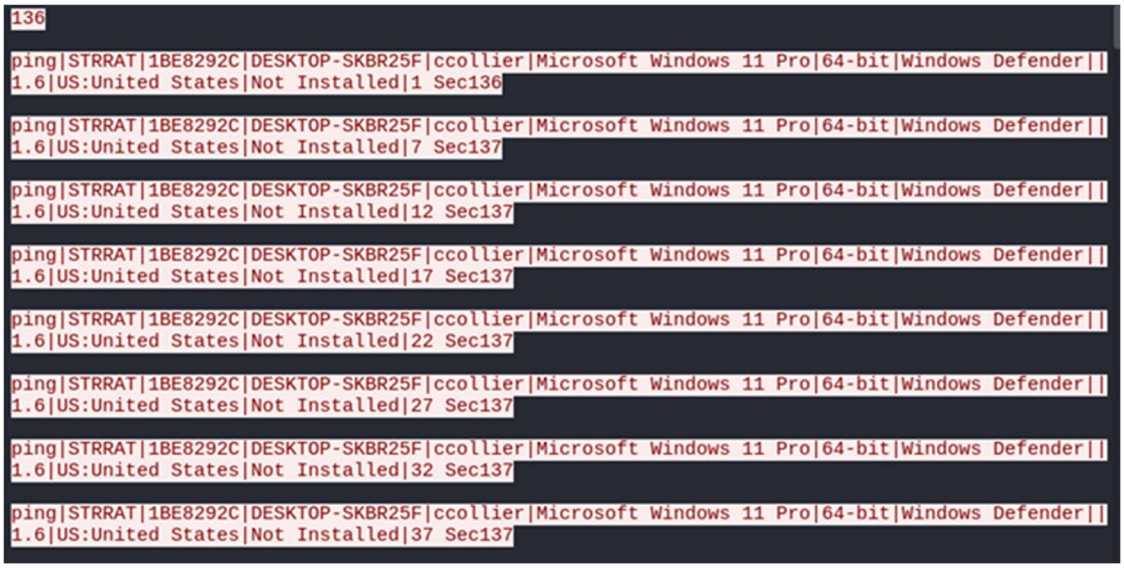


Figure 7. Repeated structured payloads visible in the TCP stream; STRRAT appears explicitly.

7 Beaconing Timestamp Analysis

To test whether the repeated STRRAT messages occurred at regular intervals, Wireshark was filtered to show packets from 172.16.1.66 to 141.98.10.79 containing ping|STRRAT. The screenshot below shows the filtered packets and their Time column. A separate Scapy script

selected outbound packets with the same payload prefix, collected capture timestamps and calculated the gaps between consecutive matches.

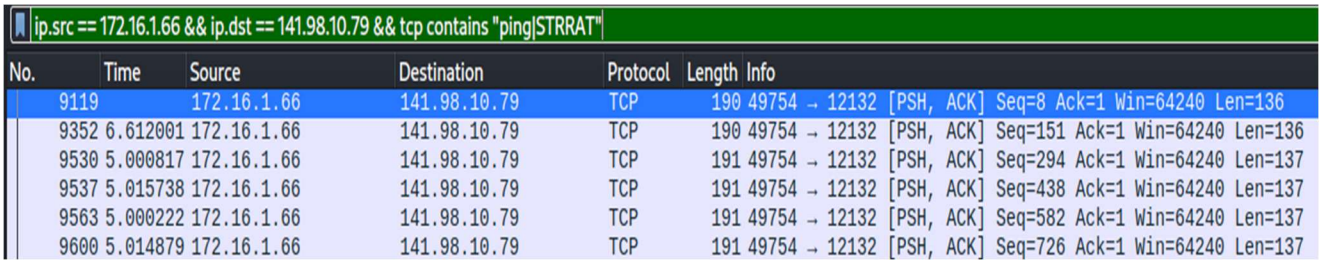


Figure 8. Wireshark view of filtered STRRAT-related traffic used for timestamp inspection.

Measured beaconing output

```
Packet 67: 2024-07-30 08:15:39.323
Packet 68: 2024-07-30 08:15:44.338
Packet 69: 2024-07-30 08:15:49.354
Packet 70: 2024-07-30 08:15:54.354
Packet 71: 2024-07-30 08:15:59.354
Packet 72: 2024-07-30 08:16:04.370
Packet 73: 2024-07-30 08:16:09.370
Packet 74: 2024-07-30 08:16:14.370
Packet 75: 2024-07-30 08:16:19.370
Packet 76: 2024-07-30 08:16:24.386
Packet 77: 2024-07-30 08:16:29.401
Packet 78: 2024-07-30 08:16:34.402
Packet 79: 2024-07-30 08:16:39.402
Packet 80: 2024-07-30 08:16:44.417
Packet 81: 2024-07-30 08:16:49.433
Packet 82: 2024-07-30 08:16:54.449
Packet 83: 2024-07-30 08:16:59.465
Packet 84: 2024-07-30 08:17:04.465
Packet 85: 2024-07-30 08:17:09.465
Packet 86: 2024-07-30 08:17:14.465
Packet 87: 2024-07-30 08:17:19.465
Packet 88: 2024-07-30 08:17:24.481
Packet 89: 2024-07-30 08:17:29.481
Packet 90: 2024-07-30 08:17:34.497
Packet 91: 2024-07-30 08:17:39.513
Packet 92: 2024-07-30 08:17:44.528
Packet 93: 2024-07-30 08:17:49.528
Packet 94: 2024-07-30 08:17:54.544
Packet 95: 2024-07-30 08:17:59.560
Packet 96: 2024-07-30 08:18:04.575
Packet 97: 2024-07-30 08:18:09.591
Packet 98: 2024-07-30 08:18:14.591
Packet 99: 2024-07-30 08:18:19.592
Packet 100: 2024-07-30 08:18:24.607
Packet 101: 2024-07-30 08:18:29.607
Packet 102: 2024-07-30 08:18:34.623
```

```
Matching packets: 102
Average interval: 5.026 sec
Median interval: 5.015 sec
Minimum interval: 4.987 sec
Maximum interval: 6.612 sec
```

```
Maximum interval: 6.612 sec
Intervals: [6.612, 5.001, 5.016, 5.0, 5.015, 5.019, 4.997, 5.001, 5.016, 5.015, 5.016, 5.018, 4.998, 5.0, 5.0, 5.016, 5.015, 5.016, 5.0, 5.016, 5.016, 5.016, 5.014, 5.002, 5.014, 4.987, 5.001, 5.0, 5.019, 5.015, 5.023, 5.007, 5.015, 5.016, 5.0, 5.016, 5.022, 5.009, 5.016, 5.011, 5.005, 5.016, 5.002, 5.017, 5.015, 5.001, 5.001, 5.016, 5.0, 5.015, 5.017, 5.016, 5.016, 5.015, 5.006, 5.01, 5.015, 5.001, 5.016, 5.016, 5.015, 5.016, 5.0, 5.016, 5.016, 5.016, 5.016, 5.016, 5.0, 5.0, 5.016, 5.0, 5.0, 5.0, 5.016, 5.015, 5.0, 5.0, 5.015, 5.016, 5.016, 5.016, 5.0, 5.0, 5.0, 5.0, 5.016, 5.0, 5.015, 5.016, 5.016, 5.0, 5.016, 5.016, 5.015, 5.016, 5.0, 5.0, 5.016, 5.0, 5.015]
```

Figure 9. Scapy timestamp listing and summary statistics for the 102 matching packets.

Timing results: 102 matching outbound packets produced 101 consecutive intervals. The average interval was 5.026 seconds, the median was 5.015 seconds, the minimum was 4.987 seconds and the maximum was 6.612 seconds. The consistent approximately five-second cadence supports automated recurring check-ins. The first interval was longer than the rest; packet timing alone cannot establish why. These are packet-level matches of the ping|STRRAT prefix; the script does not reconstruct messages split across multiple TCP segments.

8. Technical Finding: Suspected STRRAT C2 Check-ins

Attribute	Observed Evidence
Internal Host	172.16.1.66
External Peer	141.98.10.79
Transport / Port	TCP / 12132
Hostname	DESKTOP-SKBR25F
Username	ccollier
Operating System	Microsoft Windows 11 Pro (64-bit)
Security Software	Windows Defender
Payload Fragment	ping|STRRAT|...|Microsoft Windows 11 Pro|64-bit|Windows Defender|...
Behavior	102 matching outbound ping|STRRAT packets; mean interval 5.026 s; median 5.015 s
Interpretation	Regular automated check-ins consistent with suspected STRRAT C2 communication

The TCP stream revealed repeated structured messages containing the STRRAT identifier and host information, including the hostname, username, operating system, and installed security software. These messages suggest automated communication between the internal host and a remote endpoint.

The approximately five-second communication interval, combined with the distinctive STRRAT payload, provides stronger evidence of suspected command-and-control activity than the use of an uncommon destination port alone.

8.1 Host Information and User Activity Analysis

Further inspection of the TCP payload revealed that the suspicious messages contained Base64-encoded strings. Decoding selected values produced application and window titles, as shown below.

Base64-Encoded String	Decoded Value
SG9tZQ==	Home
RG9jdW1lbnRz	Documents

UGljdHVyZXM=	Pictures
UHJvZ3JhbSBNYW5hZ2Vy	Program Manager
RmFuYWQgSGVhZCBMaWdodGhvdXNlIC0gUGFpbnQ=	Fanad Head Lighthouse - Paint

These findings indicate that the suspicious communication transmitted host-identification details along with information consistent with application or active-window titles. This suggests potential monitoring of user activity by the suspected malware.

Confidence and Limitations: The Wireshark and Scapy evidence shows repeated ping|STRRAT messages, with 102 matching outbound packets and an average interval of 5.026 seconds. The captured payload also reveals system-identification details and encoded window titles. These observations support suspected automated C2 communication and possible user-activity monitoring. However, the investigation does not establish the malware executable, process tree, persistence mechanism, remote command execution, credential theft, or file exfiltration.

9. Indicators and Investigation Leads

Indicator	Value	How to Use It / Caveat
Internal IP	172.16.1.66	Host to investigate within the capture; private IP, not a global blocklist IOC
External IP	141.98.10.79	Investigate in relevant logs and threat-intelligence sources; validate before blocking
TCP Port	12132	Useful when correlated with the host pair and payload; not malicious by itself
Payload String	STRRAT	Distinctive malware-related identifier observed in the TCP stream
Payload Prefix	ping|STRRAT	Candidate signature for scoped network threat hunting
Hostname	DESKTOP-SKBR25F	Identifies the affected machine within the captured environment
Username	ccollier	User account identifier observed in the transmitted payload
Encoded Window Titles	Base64 strings representing Documents, Pictures, Paint, and other windows	Evidence consistent with transmission of user-activity metadata
Communication Pattern	Approximately five-second STRRAT check-ins	102 matching packets; 101 intervals; mean 5.026 s; median 5.015 s

10. Potential Impact and Response Recommendations

The identified traffic indicates suspected automated command-and-control communication and transmission of system-identification details to an external host. The Base64-encoded window titles further suggest possible monitoring of user activity.
If the communication is confirmed as malicious, potential security risks include unauthorized remote connectivity, exposure of host information, and monitoring of applications used on the affected system. However, the available evidence does not establish credential theft, file exfiltration, or execution of remote commands.
The following actions are recommended:

- **Host investigation:** Correlate the internal IP address, hostname, and username with endpoint logs and security monitoring records.
- **Endpoint analysis:** Examine running processes, network connections, startup entries, and persistence mechanisms to identify the process responsible for the suspicious traffic.
- **Evidence preservation:** Preserve the original PCAP and relevant endpoint artifacts with timestamps and cryptographic hashes to support reproducibility.
- **Network investigation:** Validate the external IP address using relevant security logs and reputable threat-intelligence sources before considering blocking or containment.
- **IOC-based threat hunting:** Search other relevant network captures for the same destination IP, TCP port, STRRAT payload signature, and periodic communication pattern.
- **Further payload analysis:** Inspect server-to-client communication for visible responses or commands and correlate the findings with endpoint evidence. Do not assume file exfiltration or credential theft based solely on the recurring check-in messages.

11. Conclusion

The network traffic investigation identified suspicious STRRAT-related communication between internal host 172.16.1.66 and external address 141.98.10.79 over TCP port 12132.
Python and Scapy were used to analyze 11,562 packets, identify communication patterns, and calculate beaconing intervals. The analysis identified 102 matching outbound packets with an average interval of 5.026 seconds, providing evidence of regular automated communication.
Further TCP-stream inspection revealed host-identification information, including the hostname, username, operating system, and security software. Base64-encoded application and window titles were also identified, suggesting possible user-activity monitoring.
These findings are consistent with suspected STRRAT command-and-control activity and demonstrate the effectiveness of combining automated PCAP analysis with manual Wireshark investigation. However, additional endpoint evidence would be required to establish the full extent of the compromise, including any remote command execution, credential theft, or data exfiltration.

Appendix A. Scapy Analysis Script

This script provides summary statistics and does not itself identify or classify STRRAT.

```
from scapy.all import rdpcap, IP, TCP, UDP, DNSQR
from collections import Counter

pcap_file = "malware_traffic.pcap"
print("Loading PCAP file...")
packets = rdpcap(pcap_file)
print("Total packets:", len(packets))

source_ips = []
destination_ips = []
connections = []
domains = []
ports = []

for packet in packets:
    if IP in packet:
        src = packet[IP].src
        dst = packet[IP].dst
        source_ips.append(src)
        destination_ips.append(dst)
        connections.append(f"{src} -> {dst}")
    if TCP in packet:
        ports.append(packet[TCP].dport)
    elif UDP in packet:
        ports.append(packet[UDP].dport)
    if packet.haslayer(DNSQR):
        try:
            domains.append(packet[DNSQR].qname.decode())
        except Exception:
            pass

for heading, values, limit in [
    ("TOP SOURCE IP ADDRESSES", source_ips, 10),
    ("TOP DESTINATION IP ADDRESSES", destination_ips, 10),
    ("MOST FREQUENT CONNECTIONS", connections, 10),
    ("MOST USED PORTS", ports, 10),
    ("DNS DOMAINS", domains, 20),
]:
    print("\n" + heading)
    for item, count in Counter(values).most_common(limit):
        print(item, ":", count)
```

Beaconing interval calculation — Python (Scapy)

```
from scapy.all import rdpcap, IP, TCP, Raw
from statistics import mean, median
from datetime import datetime


packets = rdpcap("2024-07-30-traffic-analysis-exercise.pcap")


timestamps = []


for pkt in packets:
    if IP in pkt and TCP in pkt and Raw in pkt:
        if (pkt[IP].src == "172.16.1.66"
            and pkt[IP].dst == "141.98.10.79"):


            payload = bytes(pkt[Raw].load)


            if b"ping|STRRAT" in payload:
                timestamps.append(float(pkt.time))
                print(
                f"Packet {len(timestamps)}: "
                f"{datetime.fromtimestamp(float(pkt.time)).strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]}"
                )


intervals = [
    timestamps[i] - timestamps[i - 1]
    for i in range(1, len(timestamps))
]


print("\nMatching packets:", len(timestamps))


if intervals:
    print("Average interval:", round(mean(intervals), 3), "sec")
```

```
    print("Median interval:", round(median(intervals), 3), "sec")
    print("Minimum interval:", round(min(intervals), 3), "sec")
    print("Maximum interval:", round(max(intervals), 3), "sec")
    print("Intervals:", [round(x, 3) for x in intervals])
else:
    print("Not enough matching packets to calculate intervals.")
```